SPORTSAPI365 · CRICKET REST CATALOGUE

Cricket API Reports

> **API:** `https://sports-api.sportsdata365.com/msdata/v1/cricket/`
>
> **Product:** Pitch Lab — a live cricket app built on the SportsAPI365 Cricket REST catalogue (39 endpoints)
>
> **Scope:** REST GET only. MCP, streaming, alerts, predictions, load testing, and third-party score comparison were not part of this work.

Contents

- 1. What we built
- 2. How we built it
- 3. How we tested
- 4. What a product needs vs what we proved
- 5. Endpoint results from the live REST run
- 6. Full catalogue (all 39 routes)
- 7. Product walkthrough — what each screen actually needed
- 8. Findings from the REST run (priority)
- 9. Agentic gap analysis — what to add, what to fix, what to improve
- 10. Endpoints that would remove the most product pain

This note is the combined record of two pieces of work that were done in sequence. First we tested the Cricket API as a customer would: live calls, a written test script, and three end-to-end demos. Then we built a real cricket product on top of those same 39 endpoints, walked every screen by hand, and used a gap harness plus coding agents to turn “the call returned 200” into “can a customer actually ship a cricket app from this feed.”

Nothing below was invented. Search returned the ids. The product only rendered what the API gave us. If a feature was not called, it is marked untested.

---

1. What we built

Pitch Lab is a small cricket product that a customer could reasonably expect from a sports data API: a home screen with live and today’s matches, a live board, a matches list with live / upcoming / results tabs, a match centre (overview, scorecard, commentary, overs, insights, odds), search, player pages, team pages, and competitions with a points table.

The product is not a production consumer app. It is the proof that the Cricket API can, or cannot, support those screens without extra work. Every product screen talks to our own BFF (`/app/cricket/...`). The BFF talks to SportsAPI365, normalises the payload, and records a gap whenever a field, relationship, or filter that the screen needs is missing, null, slow, or shaped differently from another route.

The app runs locally as two processes. The web UI is at `http://localhost:5173`. The BFF is at `http://127.0.0.1:8787`. An admin Gap Dashboard sits at `/admin/gaps` so a reviewer can see coverage, recommendations, and schema drift instead of only HTTP status codes.

The short verdict from that product is the same as the API test: the Cricket API is usable for a first product. Search, player and team pages, today's list, live match detail, head-to-head, IPL standings, and some odds all work. It is not yet the easiest live-score API to build with. A live home card cannot be built from the day's fixture list alone. Scores appear only after each match is fetched one by one. Several widgets only exist after we hide them, fall back, or assemble four routes into one scorecard.

---

2. How we built it

We treated the 39-route catalogue as the only source of truth and asked a practical question: if a customer is given this API today, can they ship a normal cricket app?

The stack is a Fastify BFF (`apps/api`) and a Vite + React UI (`apps/web`). The BFF owns the SportsAPI365 client, unwrap helpers, screen loaders, and the gap store. The UI is thin: it renders screen contracts (`/app/cricket/home`, `/matches/:id/scorecard`, `/players/:id`, and so on) and never talks to the provider directly.

That split was deliberate. List routes and detail routes do not share one schema. Search and fixtures use `items`. Player and team match lists use `events`. Seasons use `seasons`. Player, team, match, and tournament objects sit one level down (`player`, `team`, `event`, `uniqueTournament`). Status is a string on lists and an object on detail. Start time is a unix number on lists and a string on detail. A UI that consumed the raw feed would have to special-case every screen. The BFF does that once, and records the special case as a gap.

Screen loaders were written against a product contract, not against "HTTP 200." A live home card is expected to have teams, status, start time, score, innings, and overs. A player page is expected to have name plus country, role, batting/bowling style, and a photo. When the provider does not meet the contract, the loader still tries to render the screen: it may call `/event/{id}` for each live row that has no score, it may hide Insights widgets on a 404, and it may show "Tournament unavailable" when `uniqueTournament.id` is null. Those workarounds keep the product usable. They are also the evidence that the API is not yet product-complete.

Every provider call is logged with screen, component, status, time, and whether it was a fallback. Gaps are typed (missing field, optional 404, broken relationship, schema inconsistency, extra request required, and so on) and scored per screen. Recommendations on the Gap Dashboard are generated from those codes, not from a separate opinion list.

---

3. How we tested

Testing was done in four layers. Each layer answers a different question. Together they are the basis of this report.

3.1 Layer one — live REST against the 39-route catalogue

We called every route in the Cricket catalogue. The first question was simple: does the path exist, and do we get a JSON reply? A second, deeper pass then logged 52 calls plus 9 demo calls (about 61 REST requests, under a 150-call cap). Response times on the logged set were min 307 ms, average 399 ms, max 1010 ms (odds). MCP was not in this test.

Result language used throughout:

Result	Meaning
Pass / Works	200 with data a screen can show
Empty	200, nothing to show (match not started, no featured event, empty list)
Not available	404 on an optional extra. The parent match or player still exists
Fail	Wrong or unusable for the stated use (for example a finished filter empty while finished matches exist elsewhere)
Expected	400 or 404 on a bad request (missing `q`, non-numeric id, bad date)

The deeper pass followed this flow:

1. `GET /search?q=…` to resolve real player, team, and competition ids. Nothing was invented.
2. `GET /fixtures/today` and `GET /events` including `status` and paging (page 1 and page 2).
3. `GET /event/{id}` plus innings, overs, ball-by-ball, h2h, odds, period-scores, pregame-form, best-players, team-streaks, and enrichments on a live match.
4. `GET /player/{id}` plus last/next matches and IPL season statistics (Kohli and Rohit, same season).
5. `GET /team/{id}` plus featured-event, squad, form, near-events, performance, last matches, and standings-seasons.
6. IPL seasons → standings/total and standings/home.
7. Invalid `q`, id, and date, to check error behaviour.
8. Three local scripts (today, player, standings) as an end-to-end check of the same paths.

Ids used in that run, all returned by search or by a previous 200:

Record	Id	Source
Virat Kohli	55089365	search `q=Virat Kohli`
Rohit Sharma	98189805	search `q=Rohit Sharma`
India	26236865	search `q=India`
Australia	73448951	search `q=Australia`
Indian Premier League	77922875	search `q=Indian Premier League`
IPL 2026	81844788	IPL `/seasons` (first row)
Afghanistan v Bangladesh	29824257	`/fixtures/today` (in progress)

3.2 Layer two — a proper script to run the endpoints

We did not rely on one-off curls. We wrote and ran a small script kit so the same paths can be re-run from the repo.

The first kit lives with the API audit (`docs/cricket-audit-demos/` in the bot repo). It reads the base URL from `.env`.

```
source venv/bin/activate
python docs/cricket-audit-demos/demo_today.py
python docs/cricket-audit-demos/demo_player.py
python docs/cricket-audit-demos/demo_standings.py
```

Script	What it runs	What it showed
`demo_today.py`	`/fixtures/today` twice, then `/events?status=inprogress`	200; 5 matches; 4 live; scores blank on the list; second poll same ids. There is no stream — live means poll again.
`demo_player.py`	search → `/player/{id}` → `/events/last`	200; Kohli `55089365`; 3 of 553 recent matches; list key is `events`, not `items`.
`demo_standings.py`	search IPL → `/seasons` → `/standings/total`	200; IPL 2026 `81844788`; 10-row table. Seasons sit under `seasons`, not `items`.

Those scripts already distinguish empty from error. A blank score on today's list is the API, not a demo bug. A 404 on best-players is "extra not on file," not "match missing."

The product repo has a second script, `pnpm audit:phase1`, which walks routes that the first live pass had not fully covered (`/players`, `/teams`, `/tournaments`, manager, typed enrichments, team media, rankings, unique-tournaments, standings/away) and writes the same gap records the screens use. That is how catalogue coverage and product coverage were kept on one list.

A smoke check that does not need the scripts:

```
curl -sS "$MSDATA_BASE_URL/v1/cricket/search?q=Virat%20Kohli&page=1&limit=1" \
  -H "accept: application/json"
```

Expect 200 JSON with Kohli in `items`.

3.3 Layer three — manual product testing

After the BFF and UI were up, we walked the product the way a user would, not the way a catalogue would.

We opened Home and Live and checked whether live cards showed a score (`PAK 98/1`, `BAN 316/8`) without the page looking empty. We opened Matches and switched Live / Upcoming / Results to see whether the finished tab stayed empty while completed games still appeared on team pages. We opened a live match centre and moved through Overview, Scorecard, Commentary, Overs, Insights, and Odds. We searched for players, teams, and IPL. We opened Kohli, Rohit, India, Australia, and the IPL tournament page including standings.

The point of the manual pass was not "did the JSON parse." It was "does this screen work with only the payload the API gave us, or did we have to add fallbacks, hide a widget, or drop a column." That is how we caught things a catalogue-only pass could not: list rows have no scores, so Home made N+1 detail calls; match detail had a

tournament name and a null tournament id, so Match → Tournament navigation broke; innings rows had player ids but no names, so a scorecard could not be printed as a customer expects; Insights 404 looked like a missing match until we special-cased it; odds came back with selection names `1` / `2` and no prices, and the call was the slowest in the feed.

Wherever the product still rendered, we marked the screen as working with workarounds. Wherever a widget was hidden or a tab was empty, we recorded it as a gap against the endpoint that should have filled it.

3.4 Layer four — coding agents for gap analysis

The last layer was an agentic pass over the evidence, not a second set of live calls.

The flow was:

1. **Collect.** The BFF already writes every provider call and every gap (`data/calls.json`, `data/gaps.json`) while the product is used. The REST audit already has a sanitised log (`docs/cricket-audit-evidence.json`) and the endpoint tables in this report.
2. **Group.** Coding agents read those files plus the screen loaders and grouped issues by endpoint and by product impact, not by HTTP status. "200 with no score on a live row" was treated as a P1 product failure, not a pass.
3. **Compare list vs detail.** Agents compared `/fixtures/today` and `/events` against `/event/{id}` for the same match: status shape, timestamp type, tournament id, venue, score. That is how schema drift and the broken Match → Tournament link were confirmed rather than guessed.
4. **Compare player vs player.** Kohli IPL 2026 statistics returned 200. Rohit on the same tournament and season returned 404. That was classified as coverage, not as a bad player id.
5. **Map screen → endpoint.** Each product widget was mapped to the route that should feed it. Missing routes (tournament leaders, a single scorecard payload, a simple `/player/{id}/statistics`) were called out as "add," not as failed tests.
6. **Write recommendations.** The Gap Dashboard already turns gap codes into titles (`Improve /fixtures/today`, `Return empty 200 for optional extras`). Agents used that list, the manual walkthrough, and the REST tables to produce the "what to add / what to fix / what to improve" section at the end of this document.

The agent pass did not invent new endpoints or new failures. It organised what the scripts and the product already proved, and it filled the empty headings that a catalogue-only report leaves behind (best-players, team-streaks, leaders, scorecard assembly).

---

4. What a product needs vs what we proved

The live REST run had five games on today's card and four in progress, including Afghanistan vs Bangladesh (1st innings, Bangladesh 316/8). A later product walk used live T20s on the same routes (including Pakistan vs Sri Lanka) and saw the same structural gaps.

What a product needs	Ready?	What we saw
Today's matches	Yes	The day's list works.

What a product needs	Ready?	What we saw
Live scores	Partial	Live games exist. The list does not include the score. You must open each match. No live stream.
Results (completed games)	Partial	`/events?status=finished` was `total=0`. A completed ODI still showed on Australia's featured match.
Points table	Yes	IPL 2026 table returned (10 rows; RCB 18 points at the top).
Player / team pages	Yes	Kohli and India both loaded. Profile fields are thin (role `BM`, no photo, country often missing).
Player season stats	Partial	Kohli's IPL 2026 stats loaded. Rohit Sharma's stats for the same season did not (not on file).
Scorecard	Partial	Innings exist, but names, dismissals, extras, FOW, and partnerships are missing. The screen has to assemble innings + period-scores + overs + ball-by-ball.
Head to head	Yes	Loaded on the live Test.
Alerts	No	Not in the API catalogue. Not tested.
Odds / predictions	Partial	Markets come back (5 on the Test, 7 on a later live T20). Selection names were `1` / `2`. Prices were missing on the product run. Odds is also the slowest call (~1 s). No predictions route.
Typical wait	~0.3–1.0 s	Fastest logged calls ~307 ms. Odds ~1010 ms. Some product-time list calls exceeded 800 ms and were flagged slow.

You can ship against this API if the app polls and the engineers budget extra GETs per live match. You should not promise "live scores from one cheap list call," "push alerts," or "stats for every star player in a season."

---

5. Endpoint results from the live REST run

The table below is the logged live REST pass. Paths are relative to `/v1/cricket`. Times are from that run.

#	Endpoint	Input	HTTP	Time	Result	Output
1	`/search`	`q=Virat Kohli`	200	332 ms	Pass	1 hit, id 55089365
2	`/search`	`q=India`	200	378 ms	Pass	7 hits
3	`/search`	`q=Indian Premier League`	200	331 ms	Pass	1 hit, id 77922875
4	`/search`	`q=Rohit Sharma`	200	313 ms	Pass	4 hits, player id 98189805
5	`/search`	`q=Australia`	200	326 ms	Pass	4 hits, team id 73448951
6	`/search`	no `q`	400	327 ms	Expected	"q may only contain letters..."
7	`/fixtures/today`	—	200	353–756 ms	Pass	5 matches; **no scores**
8	`/events`	page=1, limit=10	200	347 ms	Pass	5 matches
9	`/events`	status=inprogress	200	336–360 ms	Pass	4 matches
10	`/events`	status=finished	200	357 ms	Empty	total=0
11	`/events`	limit=1 page=1 then page=2	200	349–358 ms	Pass	page 1 hasMore=true; page 2 returned the next row
12	`/event/29824257`	live Test	200	307–379 ms	Pass	inprogress; BAN 316/8; competition id null
13	`/event/29824257/innings`	—	200	393–432 ms	Pass	1 innings, 20 players

#	Endpoint	Input	HTTP	Time	Result	Output
14	`/event/29824257/overs`	—	200	354 ms	Pass	overs list
15	`/event/29824257/ball-by-ball`	—	200	333 ms	Pass	page 1 = 10, total = 505
16	`/event/29824257/h2h`	—	200	413 ms	Pass	teamDuel, managerDuel
17	`/event/29824257/odds`	—	200	980–1010 ms	Pass	5 markets
18	`/event/29824257/period-scores`	—	200	332 ms	Pass	periods
19	`/event/29824257/pregame-form`	—	200	394 ms	Pass	homeTeam, awayTeam
20	`/event/29824257/best-players`	live match	404	347 ms	Not available	Enrichment best_players not found
21	`/event/29824257/team-streaks`	live match	404	327 ms	Not available	Enrichment team_streaks not found
22	`/event/29824257/enrichments`	—	200	314 ms	Empty	types = 0
23	`/player/55089365`	Kohli	200	381 ms	Pass	name Virat Kohli under `player`
24	`/player/55089365/events/last`	limit=5	200	381 ms	Pass	list key `events` ; total 553
25	`/player/55089365/events/next`	limit=5	200	320 ms	Pass	list key `events`
26	`/player/55089365/…/season/81844788/statistics`	IPL 2026	200	313 ms	Pass	player, team, statistics
27	`/player/98189805/…/season/81844788/statistics`	Rohit, same IPL season	404	319 ms	Not available	No season statistics on file
28	`/player/999999999`	unknown id	404	363 ms	Expected	player not found
29	`/event/abc`	bad id	400	423 ms	Expected	numeric string expected
30	`/team/26236865`	India	200	324 ms	Pass	name India under `team`
31	`/team/26236865/featured-event`	India	200	318 ms	Empty	`event` = null
32	`/team/73448951/featured-event`	Australia	200	357 ms	Pass	finished ODI 78819074, SA 145 / AUS 322; not the live Test
33	`/team/26236865/near-events`	India	200	327 ms	Pass	previousEvent, nextEvent
34	`/team/26236865/performance`	India	200	325 ms	Pass	events, points
35	`/team/26236865/players`	India	200	426 ms	Pass	players
36	`/team/26236865/events/last`	limit=5	200	359 ms	Pass	list key `events`
37	`/team/26236865/standings-seasons`	India	200	349 ms	Pass	27 seasons; keys `uniqueTournamentSeasons` , `tournamentSeasons`
38	`/unique-tournament/77922875`	IPL	200	354 ms	Pass	Indian Premier League
39	`/unique-tournament/77922875/seasons`	IPL	200	398–609 ms	Pass	10 of 14 under `seasons` (not `items`)
40	`/unique-tournament/77922875/events`	limit=5	200	350 ms	Pass	list key `events`
41	`/unique-tournament/77922875/scheduled-events/2026-10-09`	sample calendar date	200	339 ms	Pass	list key `events`
42	`/unique-tournament/77922875/scheduled-events/not-a-date`	invalid	400	739 ms	Expected	expected YYYY-MM-DD

#	Endpoint	Input	HTTP	Time	Result	Output
43	`…/season/81844788/standings/total`	IPL 2026	200	469 ms	Pass	1 group, 10 rows. 1 RCB 18 pts … 8 CSK 12 pts
44	`…/season/81844788/standings/home`	IPL 2026	200	316 ms	Pass	1 group

Pagination: `page` is 1-based, `limit` is 1–100. Page 2 works. A huge page number returns 200 empty, not 404. Search always needs `q` (2–80 characters; letters, numbers, spaces, `'`, `.`, `-`). Missing `q` returns 400 with a character-class message, not "q is required."

HTTP behaviour we treated as reliable:

Status	Meaning	What to tell a user
200, including empty lists / `event: null`	Call worked	"Nothing for this team or match right now." Not "cricket is down."
400	Bad `q`, `abc` in an id, bad date	Fix the query. Bad date message is already clear: expected `YYYY-MM-DD`.
404	Unknown id, missing extra, no season stats	Name the missing thing. Do not retry as if the feed is down.

---

6. Full catalogue (all 39 routes)

The first catalogue pass called every route once. Combined with the depth pass and the product's phase-1 audit, this is the coverage picture. "In the bot" means the cricket bot already calls the route. The product BFF calls a wider set because the screens need it.

What this is	Endpoint	Status	Notes
Match details	`/event/{eventId}`	200 Works	Details sit under `event`. Tournament id can be null.
Ball by ball	`/event/{eventId}/ball-by-ball`	200 (empty if not started)	Live Test total=505. Useful commentary feed.
Best players (MOTM)	`/event/{eventId}/best-players`	404 Not available	Optional extra. 404 on a live match with hundreds of balls.
List of match extras	`/event/{eventId}/enrichments`	200 Works	Types list was empty.
One match extra by name	`/event/{eventId}/enrichments/{type}`	404	No extras on that match.
Head to head	`/event/{eventId}/h2h`	200 Works	Team comparison is there; coach comparison may be thin.
Innings / scorecard	`/event/{eventId}/innings`	200 Works	Use this for batting figures. Names and dismissals still missing.
Odds	`/event/{eventId}/odds`	200 (empty before start)	Slowest call. Markets exist; prices and selection names need work.
Overs	`/event/{eventId}/overs`	200 (empty if not started)	Over number, runs, wickets. No bowler or balls-in-over.
Period scores	`/event/{eventId}/period-scores`	200 (can be empty / null values)	Period row can exist with null home/away.

What this is	Endpoint	Status	Notes
Pre-game form	`/event/{eventId}/pregame-form`	200 (can be blank)	Home/away can be null on a live international.
Team streaks	`/event/{eventId}/team-streaks`	404 Not available	Same as best players — optional extra, not on this match.
All matches (feed)	`/events`	200 Works	Live filter works. Finished filter was empty.
Today's fixtures	`/fixtures/today`	200 Works	Names and status, no scores.
Coach / manager	`/manager/{managerId}`	404 unless id exists	India had no coach id. Skip unless the team payload has one.
Player profile	`/player/{playerId}`	200 Works	Profile sits under `player`.
Player's matches	`/player/{playerId}/events/{direction}`	200 Works	`last` or `next`. No player figures on the row.
Player stats in a season	`/player/{id}/unique-tournament/{tid}/season/{sid}/statistics`	200 or 404	Worked for Kohli in IPL. Rohit 404 on the same season.
All players	`/players`	200 Works	Directory is thin: name, id, role `BM`.
Search	`/search`	200 Works	Needs `q`. How we find a player or team by name.
Team profile	`/team/{teamId}`	200 Works	Profile sits under `team`. Name and country. No logo.
List of team extras	`/team/{teamId}/enrichments`	200 Works	8 extras listed (unlike the match extras list).
One team extra by name	`/team/{teamId}/enrichments/{type}`	200 Works	Bot uses `events_next`.
Team's matches	`/team/{teamId}/events/{direction}`	200 Works	`last` or `next`.
Featured match	`/team/{teamId}/featured-event`	200 Empty or finished	India null. Australia a finished ODI, not the live Test.
Team media	`/team/{teamId}/media`	200 Works	Photos.
Previous / next match	`/team/{teamId}/near-events`	200 (can be empty)	Prefer this over featured-event for "what is next."
Team form	`/team/{teamId}/performance`	200 Works	Recent form.
Squad	`/team/{teamId}/players`	200 Works	Under `players`. Role is often not enough to group a squad.
Rankings	`/team/{teamId}/rankings`	200 (list can be empty)	Empty is still a valid reply.
Seasons with a table	`/team/{teamId}/standings-seasons`	200 Works	Use this to find which season's points table to load.
Team's competitions	`/team/{teamId}/unique-tournaments`	200 Works	Useful for a team's competition list. Not a live feed.
All teams	`/teams`	200 Works	Name + country code. No logo.
All competitions	`/tournaments`	200 Works	Name only.
Competition details	`/unique-tournament/{id}`	200 Works	Under `uniqueTournament`. `currentSeasonId` can be null.
Competition matches	`/unique-tournament/{id}/events`	200 Works	Can include old seasons (IPL 2013 rows appeared on the tournament page).
Matches on a date	`/unique-tournament/{id}/scheduled-events/{date}`	200 Works	Empty list is OK. Bad date → 400 `YYYY-MM-DD`.

What this is	Endpoint	Status	Notes
Points table	`/unique-tournament/{id}/season/{sid}/standings/{type}`	200 Works	Type `total`, `home`, or `away`. NRR missing. `scoresFor` / `scoresAgainst` came back 0.
Seasons of a competition	`/unique-tournament/{id}/seasons`	200 Works	Needed before player season stats or a points table.

Not in any of these tests: MCP, streaming / alerts (not in the 39-route catalogue), predictions, comparison with Google or another score site, load testing.

---

7. Product walkthrough — what each screen actually needed

This section is the product review, written against the same endpoints as the REST tables. “Observed” is what the live API returned while Pitch Lab was running. “Needed” is what a normal cricket screen requires.

Home and Live — `GET /fixtures/today` and `GET /events`

List rows have teams, status, and start time. Live scores are not on the list. Tournament name is often missing. Venue is missing. Status is a string (`inprogress` / `notstarted`). Start time is a unix number.

`GET /fixtures/today` returned five matches. None of the four live rows had a score on the list:

Id	Match	Status on list	Score on list
29824257	Afghanistan v Bangladesh	inprogress	not present
13725236	South Africa v Australia	inprogress	not present
43247376	Boland v Easterns	notstarted	not present
49822134	India v West Indies	inprogress	not present
25643079	Pakistan v Sri Lanka	inprogress	not present

`GET /events?status=inprogress` returned the four in-progress rows. A later home screen with two live matches needed four Cricket API calls (two list calls plus two event-detail fallbacks) to show `PAK 98/1` vs `SL 0`.

Needed on every list row: `homeScore` / `awayScore` (runs, wickets, overs), current innings label, tournament id **and** tournament name, venue, and the same status object and time format as match detail. Without this, every live home page becomes an N+1 problem.

Match centre — `GET /event/{eventId}`

Scores, venue, and innings text are here (`PAK 98/1`, Rawalpindi, 1st Inning; or BAN 316/8 in 83.4). Status is an object, and the code can be `21` instead of `inprogress`. `uniqueTournament.name` is present. `uniqueTournament.id` is **null**. List rows for the same match already had a tournament id (`tournamentId` 76719630 on the AFG v BAN list row).

Needed: always set `uniqueTournament.id` when a tournament name exists; one status shape on list and detail (`{ code, type, description }`); one timestamp type (unix seconds or ISO, not both); a clear match-format field (Test

/ ODI / T20). Without the tournament id, Match → Tournament navigation breaks.

Results — `GET /events?status=finished`

`total = 0` on a day when finished matches exist on team pages (Australia's featured event was a finished ODI, SA 145 / AUS 322). A Matches → Results tab cannot stay empty while completed games are available on other routes.

Scorecard — `GET /event/{id}/innings`

On a live T20: one innings, 11 batting rows. Runs, balls, 4s, 6s are there. Player **name** is missing (only an id). No dismissal text. No extras. No fall of wickets. No partnerships. No innings team name or innings total. The live Test had 1 innings and 20 players — same shape, better volume, still not a printable scorecard.

Needed for a real scorecard: player name + player id, how out (and fielder if relevant), extras (wides, no-balls, byes, leg-byes, penalty), fall of wickets, partnerships, innings team, runs, wickets, overs.

Better still: a single `GET /event/{id}/scorecard`. Building a scorecard from innings + period-scores + overs + ball-by-ball is too much work for a standard cricket page.

Commentary — `GET /event/{id}/ball-by-ball`

Useful. Over, ball, batter, bowler, runs, and commentary text are present. Missing: score after the ball, extras type, wicket / dismissal type, fielder, timestamp.

Overs — `GET /event/{id}/overs`

Over number, runs, and wickets. Missing: balls in the over, bowler, team total after the over, innings id.

Insights — `GET /event/{id}/best-players` and `GET /event/{id}/team-streaks`

Both returned 404 on a live match (`Enrichment "best_players" was not found`) while ball-by-ball `total` was 505. The enrichments list was empty. A 404 looks like the match does not exist. If the extra is not ready, return **200** with an empty body. Keep these optional; do not make them required.

Pregame form — `GET /event/{id}/pregame-form`

`homeTeam` and `awayTeam` can be null on a live international. Needed: recent form for both sides, or a documented empty object (not a blank that looks broken).

Period scores — `GET /event/{id}/period-scores`

A period row can exist (`inning1`) with home/away values null. Needed: period totals that match the live score.

Odds — `GET /event/{id}/odds`

Markets come back. Selection names are `1` / `2`. Prices were missing on the product run. This route is also slower than the rest of the feed. Needed: market name, selection name, price, and market status. Keep it off the first match request if it stays slow.

Player profile — `GET /player/{id}`

Kohli: name, current team (RCB), role `BM`. Missing: country, batting style, bowling style, photo. Role should be a readable value (`Batter`), not `BM`. Search hits have the same problem: id and name only. No photo, country, or team.

Player season statistics

`GET /player/{id}/unique-tournament/{tid}/season/{sid}/statistics`

Kohli IPL 2026 returns 200 (matches, innings, 100s, 50s, 4s, 6s, high score). The same season for Rohit Sharma returns **404** (`No season statistics on file`). Field names also differ from what a product expects (`runsScored` instead of `runs`, `battingAverage` instead of `average`). No fielding block.

Needed: stats for every squad player in that season, or a clear "not on file" **200**; one field list (matches, innings, runs, average, strike rate, 100s, 50s, 4s, 6s, high score, wickets, economy, best figures, catches); a simple `GET /player/{id}/statistics` so the client does not have to already know tournament id + season id.

Player matches — `GET /player/{id}/events/last` and `.../next`

Match list works (Kohli has a large history). Player figures for that match are not on the row. Needed: runs / wickets for that player on each recent-match card.

Players directory — `GET /players`

Directory works (about 8,900 players). Rows are thin: name, id, role `BM`. No country, team, photo, or filters. Needed: country, team, role, photo, and query filters for country / team / role.

Team profile — `GET /team/{id}`

Name and country. No logo. No manager id on India.

Squad — `GET /team/{id}/players`

India squad returned no players in one product run; the live REST pass did return players. When players do come back, role is often not enough to group Batters / Keepers / All-rounders / Bowlers. Needed: a stable squad list with name, id, role, and photo.

Featured match — `GET /team/{id}/featured-event`

This is not the current live match. It can be empty, upcoming, or a finished game. Document it as an editorial / recommended match. Do not treat it as "live now." Live should come from `near-events` or `/events?status=inprogress`.

Directories — `GET /teams` and `GET /tournaments`

Lists work. Team row has name + country code. Tournament row has name only. No logo, category, current season, or active flag. Needed: logo, country, category, current season id.

Tournament detail — `GET /unique-tournament/{id}`

IPL name and country. `currentSeasonId` is null. `active` is null. No logo. Needed: current season id, dates, logo, category.

Tournament matches — `GET /unique-tournament/{id}/events`

Returns a mixed / old list (IPL rows from 2013 appeared on the tournament page). Needed: current-season matches by default, with season id as a filter.

Standings — `GET /unique-tournament/{id}/season/{sid}/standings/{type}`

Table works (10 IPL 2026 rows: position, team, played, won, lost, points). Missing: NRR. `scoresFor` / `scoresAginst` came back as 0. Needed: P, W, L, NRR, points — only columns that are real.

Tournament leaders — no endpoint

There is no route for Most Runs, Most Wickets, Most Sixes, Best Strike Rate, or Best Bowling. Needed: `GET /unique-tournament/{id}/season/{sid}/leaders`. Assembling this from per-player season stats is not practical, and those stats are already missing for some squad players.

Manager — `GET /manager/{id}`

404 unless a manager id exists on the team. National teams often have no manager id. Low priority. Only useful as a link from a team page.

---

8. Findings from the REST run (priority)

These are the script findings from the live REST run, kept here because the product later reproduced every one of them.

Pri	Finding	Evidence
P1	Today's list has no scores	4 live matches printed without a score; `/event/29824257` had BAN 316/8.
P1	Competition id missing on match detail	`uniqueTournament.id` null on 29824257 and on Australia featured 78819074.
P1	Status / start time differ list vs detail	List: `status` string, `statusType` null, timestamp int. Detail: `status` object, `statusType` set, timestamp string.
P1	Optional extras 404 on a live match	best-players and team-streaks 404 while ball-by-ball total=505.
P1	List array name is not always `items`	Search/fixtures use `items`. Seasons use `seasons`. Player matches use `events`.
P1	Player stats missing for one IPL squad member	Kohli IPL 2026 = 200. Rohit same season = 404.
P2	Finished filter empty	`/events?status=finished` total=0; Australia featured-event was a finished ODI.
P2	Featured match ≠ live match	India featured = null. Australia featured = finished ODI, not Test 13725236.

---

9. Agentic gap analysis — what to add, what to fix, what to improve

This is the output of the agent pass over the scripts, the call log, and the product gaps. It is split the way an API team would actually work: new payloads, broken behaviour, and quality.

9.1 Add (these routes or fields do not exist today)

1. **Scores on list rows.** `homeScore` / `awayScore`, current innings, and overs on `/fixtures/today` and `/events`. This is the single largest product cost. Four live games currently mean four extra GETs on every home refresh.
2. **A real scorecard payload.** `GET /event/{id}/scorecard` with player name + id, dismissal, extras, fall of wickets, partnerships, and innings totals. Do not ask every customer to assemble four routes.
3. **Tournament leaders.** `GET /unique-tournament/{id}/season/{sid}/leaders` for most runs, wickets, sixes, strike rate, and bowling figures. Per-player season stats cannot fill this, and they are already incomplete.
4. **A simple player statistics route.** `GET /player/{id}/statistics` so a profile page does not have to already know tournament id and season id.
5. **Match format.** Test / ODI / T20 on both list and detail. The product currently guesses from other fields.
6. **Player and team media on the thin rows.** Country, batting/bowling style, readable role, photo on `/player/{id}`, `/players`, and search hits. Logo on `/team/{id}` and `/teams`. Logo, category, current season id, and active flag on `/unique-tournament/{id}` and `/tournaments`.
7. **Player figures on recent-match cards.** Runs / wickets for that player on `/player/{id}/events/last`.
8. **NRR on standings.** And stop returning `scoresFor` / `scoresAgainst` as 0 if they are not real.
9. **Filters on `/players`.** Country, team, role. A directory of ~8,900 names without filters is not a product screen.

9.2 Fix (the route exists, but the product cannot trust it)

1. **Always set `uniqueTournament.id`** when a tournament name exists on `/event/{id}`. Keep list `tournamentId` and detail id aligned. Match → series navigation depends on this.
2. **One status object and one timestamp type** on list and detail. Prefer `{ code, type, description }` and unix seconds UTC as an integer.
3. **`/events?status=finished` must return finished matches** that already exist on team pages, or the docs must say the events feed is live/upcoming only.
4. **Optional extras must not 404 on a valid match.** `best-players`, `team-streaks`, and `/enrichments/{type}` should return 200 with an empty body. A 404 cannot be distinguished from "this match id is wrong."
5. **Season statistics coverage.** Same tournament + season should not 200 for Kohli and 404 for Rohit without a documented "not on file" 200. Field names should be one list: `runs`, `average`, not `runsScored` / `battingAverage`.
6. **Tournament events should default to the current season.** IPL 2013 rows on the tournament page are stale data, not a fixtures feed.
7. **Squad list must be stable.** India returning no players on one product run and a list on the REST run is not shippable. Include name, id, role, photo.
8. **Period scores must match the live score.** A period row with null home/away is worse than no period row.

9. **Pregame form should be form or an explicit empty object**, not null teams on a live international.

9.3 Improve (usable, but costly or confusing)

1. **Unwrap keys.** Search/fixtures/events use `items`. Seasons use `seasons`. Player/team/tournament match lists use `events`. Standings-seasons use `uniqueTournamentSeasons` / `tournamentSeasons`. Standings rows sit at `standings[0].rows`. One unwrap helper belongs in every official example.
2. **Nested wrappers.** Read `body.player`, `body.team`, `body.event`, `body.uniqueTournament`. Examples that read `id` at the top will fail. Show the `{ "event": null }` featured-event case in the docs.
3. **Document `featured-event` as editorial.** India null and Australia's finished ODI are both valid under that reading. They are invalid if a client treats the route as "live now."
4. **Odds.** Market name, selection name (not `1` / `2`), price, market status. Keep the call off the first match payload while it stays near 1 second.
5. **Ball-by-ball.** Add score after the ball, extras type, wicket type, fielder, timestamp.
6. **Overs.** Add balls in the over, bowler, team total after the over, innings id.
7. **Search errors.** Missing `q` should say `q` is required, not only list allowed characters.
8. **Role codes.** `BM` is not a product string. Map to Batter / Bowler / All-rounder / Wicket-keeper.
9. **Latency.** Publish typical times. Do not put odds (or any call that regularly exceeds ~800 ms) on the first home or match request.
10. **Polling contract.** There is no stream. Two polls of today's list in the demo returned the same five ids. Say so in the docs, including a suggested poll interval.

---

10. Endpoints that would remove the most product pain

If the API team can only do a short list, this is the order that would have changed Pitch Lab the most:

1. Scores, overs, innings, and tournament id/name on `/fixtures/today` and `/events`.
2. `uniqueTournament.id` always set on `/event/{id}`.
3. One status + time schema on list and detail.
4. A real scorecard payload (names, dismissals, extras, FOW, partnerships).
5. Working `/events?status=finished`.
6. Season stats for every squad player, with one field list.
7. Tournament leaders.
8. 200-empty instead of 404 for optional match extras.

Product assessment from live REST calls against the Cricket catalogue and a working Pitch Lab product.